

8교시: 포트 충돌, 정리, 주의할 점 확인

수업 목표

- 같은 host port를 두 PostgreSQL container가 동시에 사용할 수 없음을 실험으로 확인한다.
- port mapping을 바꾸면 같은 container port 5432를 가진 DB가 동시에 실행될 수 있음을 설명한다.
- container와 volume 정리 기준을 구분하고 Day 2 준비 상태를 확인한다.

강의 전개

이 교시는 의도적으로 실패를 만든 뒤 그 실패를 확인 지점으로 읽는 시간이다. 앞 교시에서 PostgreSQL 16과 18은 서로 다른 host port를 사용했기 때문에 동시에 실행됐다. 이제 같은 host port를 두 container에 publish하려고 하면 왜 실패하는지 직접 확인한다.

핵심은 container 내부 port와 host port를 분리하는 것이다. container 내부에서는 PostgreSQL 16도 5432, PostgreSQL 18도 5432를 사용할 수 있다. 두 process가 서로 다른 container namespace 안에 있기 때문이다. 하지만 host machine의 localhost:15432는 하나의 network endpoint다. 이미 paperclip-pg16이 그 endpoint를 사용 중이면 두 번째 container가 같은 host port를 잡을 수 없다.

실패 메시지는 수업의 정답 일부다. port is already allocated, Bind for ... failed, address already in use 같은 표현은 Docker version과 OS에 따라 조금씩 다를 수 있다. 학생은 문구를 그대로 외우는 대신 "같은 host IP/port 조합을 이미 사용 중이다"라는 원인을 구분한다.

정리 단계에서는 container와 volume을 구분한다. container를 삭제하면 process와 container writable layer는 사라지지만 named volume은 남을 수 있다. 실습 DB 데이터를 유지하고 싶다면 volume을 남긴다. 완전히 초기화하고 싶다면 volume까지 삭제한다. 이 구분은 Day 2 이후 Dockerfile과 Compose를 배울 때도 계속 이어진다.

마지막 확인 지점은 Day 1 전체를 닫는 운영 확인이다. Docker 설치 경로, version 확인, Docker 개념 요약, 로컬 PostgreSQL 처리 결정, PostgreSQL 16/18 port와 version 결과, 의도적 port conflict error, cleanup 결과가 모두 들어가야 한다. 이 확인이 있으면 Day 2에서 build와 Dockerfile로 넘어갈 때 남은 blocker를 빠르게 분리할 수 있다.

Hands-on 1: 같은 host port 충돌 만들기

먼저 paperclip-pg16이 15432:5432로 실행 중인지 확인한다.

```
docker ps --filter name=paperclip-pg16
```

이제 PostgreSQL 18도 같은 host port 15432를 쓰도록 일부러 실행해 본다.

```
docker run -d \
  --name paperclip-pg18-conflict \
  -e POSTGRES_PASSWORD=postgres \
  -p 15432:5432 \
  postgres:18
```

기대되는 실패

정상적인 실패는 아래 유형이다.

```
Bind for 0.0.0.0:15432 failed: port is already allocated
```

문구는 Docker version과 OS에 따라 다를 수 있다. 핵심은 host port 15432를 이미 다른 process/container가 사용 중이라 publish할 수 없다는 점이다.

실패 container가 생성되어 남아 있으면 정리한다.

```
docker rm paperclip-pg18-conflict
```

충돌 원인 분석

container 내부에서는 PostgreSQL 16과 18이 모두 5432를 써도 된다. 서로 다른 container namespace 안에서 각자 listen하기 때문이다. 하지만 host에서 외부 접속을 받을 port는 host machine의 network 자원이다. host의 같은 IP/port 조합은 한 process 또는 한 published port만 사용할 수 있다.

구분	PostgreSQL 16	PostgreSQL 18
container port	5432	5432
정상 host port	15432	15433
충돌 host port	15432	15432
결과	실행 가능	두 번째 container publish 실패

정상 port mapping 재확인

정상 상태를 다시 확인한다.

```
docker ps --filter name=paperclip-pg
docker exec paperclip-pg16 psql -U postgres -d paperclip -c "SELECT
version();"
docker exec paperclip-pg18 psql -U postgres -d paperclip -c "SELECT
version();"
```

host psql client가 있는 학생은 localhost:15432, localhost:15433 접속도 함께 확인한다.

```
PGPASSWORD=postgres psql -h localhost -p 15432 -U postgres -d paperclip -c
"SELECT current_setting('server_version');"
PGPASSWORD=postgres psql -h localhost -p 15433 -U postgres -d paperclip -c
"SELECT current_setting('server_version');"

```

container/volume cleanup

실습 마감에는 두 가지 선택지가 있다.

선택	명령	의미
container만 정리	<code>docker stop, docker rm</code>	DB process는 사라지지만 named volume data는 남을 수 있음
container와 실습 volume 정리	<code>docker volume rm</code> 추가	오늘 실습 DB data까지 삭제

오늘 실습 데이터가 필요 없다면 모두 정리한다.

```
docker stop paperclip-pg16 paperclip-pg18
docker rm paperclip-pg16 paperclip-pg18
docker volume rm paperclip-pg16-data paperclip-pg18-data
docker ps -a --filter name=paperclip-pg
docker volume ls | grep paperclip-pg

```

`docker volume rm`은 데이터를 삭제한다. 개인 DB나 중요한 데이터가 연결된 volume에는 사용하지 않는다.

Day 1 주의할 점 정리

- 같은 host port를 두 container가 동시에 사용할 수 없다. port is already allocated 계열 오류는 정상적인 충돌 신호다.
- Docker 설치, PostgreSQL container 실행, port conflict, cleanup은 서로 다른 문제다. 하나가 실패했다고 나머지 개념까지 실패한 것은 아니다.
- Cleanup할 때 container와 volume을 구분한다. container 삭제는 process 정리이고, volume 삭제는 data 삭제다.
- Day 2로 넘어가기 전에 남아 있는 blocker가 설치 문제인지, port 문제인지, DB 접속 문제인지 구분한다.

확인 기준

기준	확인 지점
port 충돌 이해	같은 host port를 두 container가 동시에 사용할 수 없음을 error로 확인했다.
병렬 실행 이해	15432:5432, 15433:5432 mapping을 설명했다.
version 확인	16/18 query 결과를 분리해 확인했다.
cleanup	container와 volume cleanup 범위를 구분했다.
안전	기존 로컬 PostgreSQL data 삭제 위험을 확인했다.

공식 근거 링크

- Docker Docs: Publishing and exposing ports, <https://docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/>
- Docker Docs: docker container rm, <https://docs.docker.com/reference/cli/docker/container/rm/>
- Docker Docs: docker volume rm, <https://docs.docker.com/reference/cli/docker/volume/rm/>
- PostgreSQL official image README: <https://github.com/docker-library/docs/blob/master/postgres/README.md>

다음 연결

Day 2는 오늘 실행한 container 개념을 Dockerfile과 image build로 확장한다. 오늘은 공식 image를 pull해서 실행했고, 내일부터는 실행 조건을 직접 Dockerfile로 고정한다.